Python – Summary

Built-in Functions

abs(x) ➡ num
all(iterable) ➡ bool
any(iterable) ➡ bool
ascii(object) ➡ str
bin(x) ➡ str
bool([x]) ➡ bool
breakpoint(*args, **kws)
bytearray([source[,encoding[,errors]]]) ➡ bytearray
bytes([source[, encoding[, errors]]]) ➡ bytes
callable(object) ➡ bool
chr(i) ➡ str
@classmethod
compile★
complex([real[, imag]]) ➡ complex
delattr(object, name)
dict(**kwarg) ➡ dict
dict(mapping, **kwarg) ➡ dict
dict(iterable, **kwarg) ➡ dict
dir([object]) ➡ list
divmod(a, b) ➡ num
enumerate(iterable, start=0) ➡ enumerate object
eval(expression[, globals[, locals]]) ➡ object
exec(object[, globals[, locals]])
filter(function, iterable)
float([x]) ➡ float
format(value[, format_spec]) ➡ str
frozenset([iterable]) ➡ frozenset
getattr(object, name[, default]) ➡ object
globals() ➡ dict
hasattr(object, name) ➡ bool
hash(object) ➡ int
help([object])
hex(x) ➡ str
id(object) ➡ int
input([prompt]) ➡ str
int([x]) ➡ class
int(x, base=10) ➡ int
isinstance(object, classinfo) ➡ bool
issubclass(class, classinfo) ➡ bool
iter(object[, sentinel]) ➡ iterator
len(s) ➡ int
list([iterable]) ➡ list
locals() ➡ dict
map(function, iterable, ...) ➡ iterator
max(iterable, *[, key, default]) ➡ object
max(arg1, arg2, *args[, key]) ➡ object
min(iterable, *[, key, default]) ➡ object
min(arg1, arg2, *args[, key]) ➡ object
next(iterator[, default]) ➡ object
object ➡ object
oct(x) ➡ str
open★
ord(c) ➡ int
pow(base, exp[, mod]) ➡ num
print★
property★
range(stop) ➡ sequence object
range(start, stop[, step]) ➡ sequence object
repr(object) ➡ str
reversed(seq) ➡ iterator
round(number[, ndigits]) ➡ num
set([iterable]) ➡ set
setattr(object, name, value)
slice(stop) ➡ slice object
slice(start, stop[, step]) ➡ slice object
sorted(iterable, *, key=None, reverse=False) ➡ list
@staticmethod
str(object=) ➡ str
str★
sum(iterable, /, start=0) ➡ num
super([type[, object-or-type]]) ➡ proxy object
tuple([iterable]) ➡ tup
type(object) ➡ type object
type(name, bases, dict) ➡ type
vars([object]) ➡ __dict__ attribute
zip(*iterables) ➡ iterator
__import__★

★

__import__(name, globals=None, locals=None, fromlist=(), level=0)
compile(source, filename, mode, flags=0, dont_inherit=False, optimize=-1) ➡ AST object
print(*objects, sep=, end="\n", file=sys.stdout, flush=False)
property(fget=None, fset=None, fdel=None, doc=None) ➡ property attribute
str(object=b, encoding="utf-8", errors="strict") ➡ str

open(file, mode="r", buffering=-1, encoding=None, errors=None, newline=None, ...) ➡ file object (file)
mode = "r" | "w" | "x" | "a" | "b" | "t" | "+"

file.close()
file.read(size) ➡ str | bytes
file.readlines(hint) ➡ list
file.writelines(list)
file.readline(size) ➡ str
file.write(str | bytes)

Dict Methods

dict.clear()
dict.copy() ➡ dict
dict.fromkeys(iterable[, value]) ➡ dict
dict.get(key[, default]) ➡ object
dict.items() ➡ dict_items
dict.keys() ➡ dict_keys
dict.pop(key[, default]) ➡ object
dict.popitem() ➡ tup
dict.setdefault(key[, default]) ➡ object
dict.update([other])
dict.values() ➡ dict_values

List Methods

```
list.append(x)
list.clear()
list.copy() ➡ list
list.count(x) ➡ int
list.extend(iterable)
list.index(x[, start[, end]]) ➡ int
list.insert(i, x)
list.pop([i]) ➡ object
list.remove(x)
list.reverse()
list.sort(key=None, reverse=False)
```

String Methods

Testing

```
str.endswith(suffix[, start[, end]]) ➡ bool
str.isalnum() ➡ bool
str.isalpha() ➡ bool
str.isascii() ➡ bool
str.isdecimal() ➡ bool
str.isdigit() ➡ bool
str.isidentifier() ➡ bool
str.islower() ➡ bool
str.isnumeric() ➡ bool
str.isspace() ➡ bool
str.istitle() ➡ bool
str.isupper() ➡ bool
str.isprintable() ➡ bool
str.startswith(prefix[, start[, end]]) ➡ bool
```

Formatting

```
str.center(width[, fillchar]) ➡ str
str.format(*args, **kwargs) ➡ str
str.format_map(mapping) ➡ str
str.rjust(width[, fillchar]) ➡ str
str.ljust(width[, fillchar]) ➡ str
str.zfill(width) ➡ str
```

Transformation

```
str.encode★
str.expandtabs(tabsize=8) ➡ str
str.lstrip([chars]) ➡ str
str.replace(old, new[, count]) ➡ str
str.rstrip([chars]) ➡ str
str.strip([chars]) ➡ str
```

★

```
str.encode(encoding="utf-8", errors="strict") ➡ bytes object
```

Upper and Lower Cases

```
str.capitalize() ➡ str
str.casefold() ➡ str
str.lower() ➡ str
str.swapcase() ➡ str
str.title() ➡ str
str.upper() ➡ str
```

Split & Combine

```
str.join(iterable) ➡ str
str.partition(sep) ➡ tup
str.rpartition(sep) ➡ tup
str.split(sep=None, maxsplit=-1) ➡ list
str.rsplit(sep=None, maxsplit=-1) ➡ list
str.splitlines([keepends]) ➡ list
```

Searching

```
str.count(sub[, start[, end]]) ➡ int
str.find(sub[, start[, end]]) ➡ int
str.index(sub[, start[, end]]) ➡ int
str.rfind(sub[, start[, end]]) ➡ int
str.rindex(sub[, start[, end]]) ➡ int
```

String Formatting

`{<f_expression> ["="] [!<conversion>] [:<format_spec>]}`

conversion	s: with str(), r: with repr(), a: with ascii(),
format_spec	[[fill]align][sign][#][0][width][group][.precision][type]
fill	<any character> (except "{" and "}")
align	"<" | ">" | "=" | "^"
sign	"+" | "-" | " "
width	positive integer
group	"_" | ","
precision	positive integer
type	"b" | "c" | "d" | "e" | "E" | "f" | "F" | "g" | "G" | "n" | "o" | "s" | "x" | "X" | "%"

Set Methods

```
set.add(elem)
set.clear()
set.copy() ➡ set
set.difference(*others) ➡ set
set.difference_update(*others)
set.discard(elem)
set.intersection(*others) ➡ set
set.intersection_update(*others)
set.isdisjoint(bool)
set.issubset(other) ➡ bool
set.issuperset(other) ➡ bool
set.pop() ➡ object
set.remove(elem)
set.symmetric_difference(other) ➡ set
set.symmetric_difference_update(other)
set.union(*others) ➡ set
set.update(*others)
```

NumPy (`import numpy as np`)

Legende: "➡ndarray" = copy | "➡ndarray ◇" = view | "➡ndarray (◇)" = view/copy

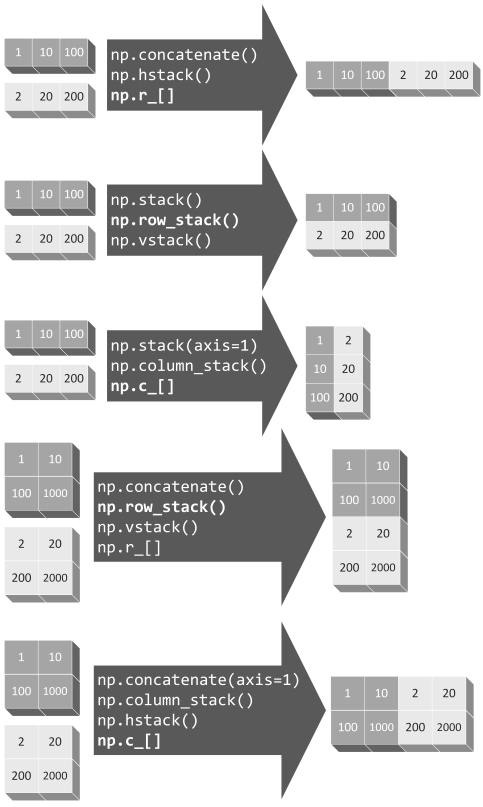
Array Creation

```
np.array(object, dtype=None, *, copy=True, order="K", subok=False, ndmin=0) ➡ndarray
np.arange([start,] stop[, step,]dtype=None) ➡ndarray
np.asarray(a[, dtype, order]) ➡ndarray (◇)
np.copy(a, order="K", subok=False) ➡ndarray
np.empty(shape, dtype=float, order="C") ➡int ➡ndarray
np.empty_like(prototype, dtype=None, order="K", subok=True, shape=None) ➡ndarray
np.eye(N, M=None, k=0, dtype=<class "float">, order="C") ➡ndarray
np.full(shape, fill_value, dtype=None, order="C") ➡ndarray
np.full_like(a, fill_value, dtype=None, order="K", subok=True, shape=None) ➡ndarray
np.fromfile(file, dtype=float, count=-1, sep=, offset=0) ➡ndarray
np.fromstring(string, dtype=float, count=-1, sep=) ➡ndarray
np.identity(n, dtype=None) ➡ndarray
np.linspace(start, stop, num=50, endpoint=True, retstep=False, dtype=None, axis=0) ➡ndarray
np.loadtxt(fname, dtype=<class "float">, comments="#", delimiter=None, converters=None, skiprows=0,
   usecols=None, unpack=False, ndmin=0, encoding="bytes", max_rows=None) ➡ndarray
np.logspace(start, stop, num=50, endpoint=True, base=10.0, dtype=None, axis=0) ➡ndarray
np.meshgrid(*xi, copy=True, sparse=False, indexing=*y") ➡ndarray
np.ones(shape, dtype=None, order="C") ➡ndarray
np.ones_like(a, dtype=None, order="K", subok=True, shape=None) ➡ndarray
np.random.rand(d0, d1, ..., dn) ➡ndarray
np.random.randn(d0, d1, ...,dn) ➡ndarray
np.random.randint(low, high=None, size=None, dtype=int) ➡ndarray
np.zeros(shape, dtype=float, order="C") ➡ndarray
np.zeros_like(a, dtype=None, order="K", subok=True, shape=None) ➡ndarray
```

Array Manipulation

```
numpy.append(arr, values, axis=None) ➡ndarray
np.array_split(ary, indices_or_sections, axis=0) ➡list
np.atleast_1d(*arys) ➡ndarray
np.atleast_2d(*arys) ➡ndarray
np.atleast_3d(*arys) ➡ndarray
np.c_[...] ➡ndarray
np.column_stack(tup) ➡ndarray
np.concatenate((a1, a2, ...), axis=0, out=None) ➡ndarray
np.dstack(tup) ➡ndarray
np.expand_dims(a, axis) ➡ndarray ◇
np.ndarray.flatten([order]) ➡ndarray
np.hstack(tup) ➡ndarray
np.r_[...] ➡ndarray
np.ravel(a, order="C") ➡ndarray ➡array_like (◇)
np.repeat(a, repeats, axis=None) ➡ndarray
np.reshape(a, newshape, order="C") ➡ndarray (◇)
np.resize(a, newshape) ➡ndarray
np.roll(a, shift, axis=None) ➡ndarray
np.row_stack(tup) ➡ndarray
np.split(ary, indices_or_sections, axis=0) ➡list ◇
np.stack(arrays, axis=0, out=None) ➡ndarray
np.tile(A, reps) ➡ndarray
np.transpose(a, axes=None) ➡ndarray
np.ndarray.T ➡ndarray (◇)
np.unravel_index(indices, shape, order="C") ➡tup
np.vstack(tup) ➡ndarray
```

Description	1D→1D	1D→2D	2D→2D	2D→3D
`np.concatenate(arrays, axis=0, out=None)` Join a sequence of arrays along an **existing** axis.	0[1]	–	0/1	–
`np.stack(arrays, axis=0, out=None)` Join a sequence of arrays along a **new** axis.	–	0/1	–	0/1/2
`np.column_stack(tup)` Stack 1-D arrays as columns into a 2-D array.	–	1	1	–
`np.row_stack(tup)` Stack arrays in sequence vertically (row wise).	–	0	0	–
`np.hstack(tup)` Stack arrays in sequence horizontally (col wise).	0	–	1	–
`np.vstack(tup)` Stack arrays in sequence vertically (row wise).	–	0	0	–
`np.c_[]` Translates slice objects to concat. along the second axis.	–	1	1	–
`np.r_[]` Translates slice objects to concat. along the first axis.	0	–	0	–



[1] This number specifies the axis over which the arrays are stacked.

Mathematical Functions

```
np.arccos(x, /, out=None, *, where=True, casting='same_kind', order='K', dtype=None, subok=True[, signature, extobj]) ➡ ndarray
np.arcsin(x, /, out=None, *, where=True, casting='same_kind', order='K', dtype=None, subok=True[, signature, extobj]) ➡ ndarray
np.arctan(x, /, out=None, *, where=True, casting='same_kind', order='K', dtype=None, subok=True[, signature, extobj]) ➡ ndarray
np.arctan2(x1, x2, /, out=None, *, where=True, casting='same_kind', order='K', dtype=None, subok=True[, signature, extobj]) ➡ ndarray
np.ceil(x, /, out=None, *, where=True, casting='same_kind', order='K', dtype=None, subok=True[, signature, extobj]) ➡ ndarray|scalar
np.cos(x, /, out=None, *, where=True, casting='same_kind', order='K', dtype=None, subok=True[, signature, extobj]) ➡ ndarray
np.cumsum(a, axis=None, dtype=None, out=None) ➡ ndarray
np.degrees(x, /, out=None, *, where=True, casting='same_kind', order='K', dtype=None, subok=True[, signature, extobj]) ➡ ndarray
np.diff(a, n=1, axis=-1, prepend=<no value>, append=<no value>) ➡ ndarray
np.exp(x, /, out=None, *, where=True, casting='same_kind', order='K', dtype=None, subok=True[, signature, extobj]) ➡ ndarray|scalar
np.floor(x, /, out=None, *, where=True, casting='same_kind', order='K', dtype=None, subok=True[, signature, extobj]) ➡ ndarray|scalar
np.log(x, /, out=None, *, where=True, casting='same_kind', order='K', dtype=None, subok=True[, signature, extobj]) ➡ ndarray
np.log10(x, /, out=None, *, where=True, casting='same_kind', order='K', dtype=None, subok=True[, signature, extobj]) ➡ ndarray
np.radians(x, /, out=None, *, where=True, casting='same_kind', order='K', dtype=None, subok=True[, signature, extobj]) ➡ ndarray
np.sin(x, /, out=None, *, where=True, casting='same_kind', order='K', dtype=None, subok=True[, signature, extobj]) ➡ ndarray
np.sum(a, axis=None, dtype=None, out=None, keepdims=<no value>, initial=<no value>, where=<no value>) ➡ ndarray
np.sqrt(x, /, out=None, *, where=True, casting='same_kind', order='K', dtype=None, subok=True[, signature, extobj]) ➡ ndarray
np.tan(x, /, out=None, *, where=True, casting='same_kind', order='K', dtype=None, subok=True[, signature, extobj]) ➡ ndarray
```

Linear Algebra

```
np.cross(a, b, axisa=-1, axisb=-1, axisc=-1, axis=None) ➡ ndarray
np.dot(a, b, out=None) ➡ ndarray
np.linalg.det(a) ➡ array_like
np.linalg.eig(a) ➡ ndarray
np.inner(a, b) ➡ ndarray
np.linalg.inv(a) ➡ ndarray
np.matmul(x1, x2, **kwargs) ➡ ndarray
np.linalg.matrix_rank(M, tol=None, hermitian=False) ➡ array_like
np.linalg.norm(x, ord=None, axis=None, keepdims=False) ➡ float|ndarray
np.outer(a, b, out=None) ➡ ndarray
np.linalg.solve(a, b) ➡ ndarray
M @ v ➡ ndarray
np.trace(a, offset=0, axis1=0, axis2=1, dtype=None, out=None) ➡ ndarray
```

Matplotlib (import matplotlib.pyplot as plt)

Creation

```
plt.figure(num=None, figsize=None, dpi=None, facecolor=None, edgecolor=None, frameon=True,
    FigureClass=<class "matplotlib.figure.Figure">, clear=False, **kwargs) ➡ Figure
plt.subplot(*args, **kwargs) ➡ AxesSubplot
plt.subplots(nrows=1, ncols=1, *, sharex=False, sharey=False, squeeze=True, subplot_kw=None,
    gridspec_kw=None, **fig_kw) ➡ Figure, axes.Axes
plt.twinx(ax=None) ➡ AxesSubplot
plt.twiny(ax=None) ➡ AxesSubplot
plt.tight_layout(*, pad=1.08, h_pad=None, w_pad=None, rect=None)
plt.savefig(*args, **kwargs)
plt.show(*, block=None)
```

Drawing

```
plt.annotate(text, xy, *args, **kwargs) ➡ Annotation
plt.arrow(x, y, dx, dy, **kwargs) ➡ FancyArrow
plt.contour(*args, data=None, **kwargs) ➡ QuadContourSet
plt.contourf(*args, data=None, **kwargs) ➡ QuadContourSet
plt.loglog(*args, **kwargs) ➡ list of Line2D objects
plt.plot(*args, scalex=True, scaley=True, data=None, **kwargs) ➡ list of Line2D objects
plt.scatter(x, y, s=None, c=None, marker=None, cmap=None, norm=None, vmin=None, vmax=None, alpha=None,
    linewidths=None, verts=<deprecated parameter>, edgecolors=None, *, plotnonfinite=False, data=None,
    **kwargs) ➡ PathCollection
plt.semilogx(*args, **kwargs) ➡ list of Line2D objects
plt.semilogy(*args, **kwargs) ➡ list of Line2D objects
plt.stem(*args, linefmt=None, markerfmt=None, basefmt=None, bottom=0, label=None,
    use_line_collection=True, data=None) ➡ StemContainer
plt.step(x, y, *args, where="pre", data=None, **kwargs) ➡ list of Line2D objects
plt.streamplot(x, y, u, v, density=1, linewidth=None, color=None, cmap=None, norm=None, arrowsize=1,
    arrowstyle="-|>", minlength=0.1, transform=None, zorder=None, start_points=None, maxlength=4.0,
    integration_direction="both", *, data=None) ➡ StreamplotSet
plt.text(x, y, s, fontdict=None, **kwargs) ➡ Text
```

Decoration

```
plt.axis(*args, emit=True, **kwargs) ➡ Annotation
plt.axhline(y=0, xmin=0, xmax=1, **kwargs) ➡ Line2D
plt.axvline(x=0, ymin=0, ymax=1, **kwargs) ➡ Line2D
plt.colorbar(mappable=None, cax=None, ax=None, **kw) ➡ Colorbar
plt.grid(b=None, which="major", axis="both", **kwargs)
plt.legend(*args, **kwargs)
plt.margins(*margins, x=None, y=None, tight=True) ➡ tup
plt.suptitle(t, **kwargs) ➡ Text
plt.title(label, fontdict=None, loc=None, pad=None, *, y=None, **kwargs) ➡ Text
plt.xlabel(xlabel, fontdict=None, labelpad=None, *, loc=None, **kwargs)
plt.xlim(*args, **kwargs) ➡ tup
plt.xticks(ticks=None, labels=None, **kwargs) ➡ tup
plt.ylabel(ylabel, fontdict=None, labelpad=None, *, loc=None, **kwargs)
plt.ylim(*args, **kwargs) ➡ tup
plt.yticks(ticks=None, labels=None, **kwargs) ➡ tup
```

Decoration OOP (The names of the listed OOP methods differ from the non-OOP method names.)

```
Axes.set_title(self, label, fontdict=None, loc=None, pad=None, *, y=None, **kwargs) ➡ Text
Axes.set_xlabel(self, xlabel, fontdict=None, labelpad=None, *, loc=None, **kwargs)
Axes.set_xlim(self, left=None, right=None, emit=True, auto=False, *, xmin=None, xmax=None) ➡ tup
Axes.set_xticks(self, ticks, *, minor=False) ➡ tup
Axes.set_ylabel(self, ylabel, fontdict=None, labelpad=None, *, loc=None, **kwargs)
Axes.set_ylim(self, bottom=None, top=None, emit=True, auto=False, *, ymin=None, ymax=None) ➡ tup
Axes.set_yticks(self, ticks, *, minor=False) ➡ tup
```

Linestyle		Marker						Color		
"-"	"solid"	"."	•	"o"	●	"v"	▼	"b","blue"	"g","green"	"r","red"
":"	"dotted"	"s"	■	"p"	⬟	">"	▶	"c","cyan"	"m","magenta"	"k","black"
"--"	"dashed"	"*"	★	"d","D"	♦	"^"	▲	"w","white"	"y","yellow"	"#0F0F0F"
"-."	"dashdot"	"x","X"	×	"+","P"	+	"<"	◀	(0.0, 0.5, 1.0)		